Suivi Alumni EIGSI

Rapport de séance 5

Évaluation du MVP, retour d'expérience et évolution du processus

Ziad Harrasse et Ahmed Cherif
28 octobre 2026

1. Démonstration et évaluation du To-Be

Le projet répond au besoin de centraliser les parcours des diplômés, de fiabiliser leur actualisation et de fournir des indicateurs utiles au Service Alumni et à la Direction. Le MVP couvre la saisie, le consentement, le contrôle humain, la publication dans l'annuaire et le pilotage.

Parcours de démonstration

Acteur	Parcours à démontrer	Résultat attendu
Alumni	Créer le compte, compléter la fiche, consentir et soumettre	Fiche persistée ; accès limité à son propre profil
Service Alumni	Examiner l'unicité, demander une correction puis valider	Décision motivée, auteur et date conservés
Direction	Consulter l'annuaire et le tableau de bord	Profils publiables et indicateurs issus de l'API
Administrateur	Importer un CSV, vérifier le bilan et exporter	Droits contrôlés, erreurs expliquées et export journalisé

Illustration du MVP connecté

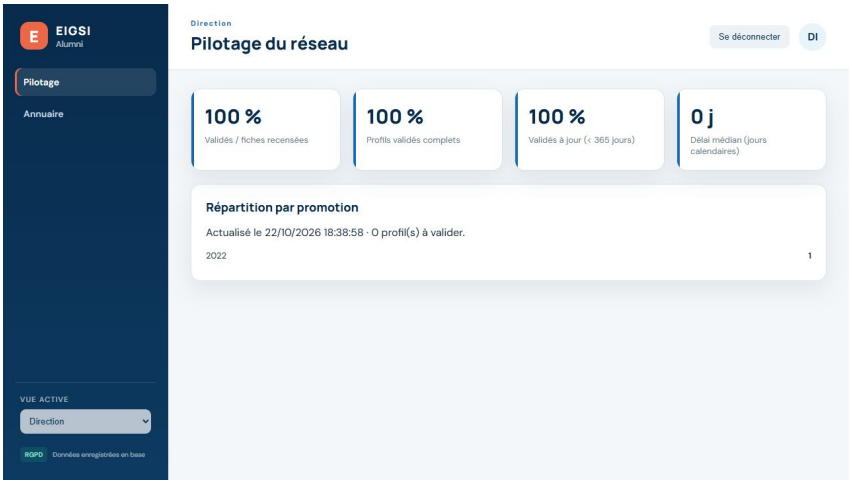


Figure 1. Capture du tableau de bord présente dans le dépôt. Données fictives de démonstration ; cette image ne constitue pas une mesure de performance métier.

État de la recette : aucun compte-rendu de démonstration aux acteurs métiers ni retour utilisateur exploitable n'est disponible pour ce rapport. L'adéquation est donc analysée à partir du cadrage, du code et des tests. L'acceptation client reste à documenter.

1. Évaluation et KPI

Adéquation au cadrage et écarts de réalisation

Attente initiale	Réalisation actuelle	Écart et suite à donner
Base unique et profils fiables	API FastAPI, persistance SQL, contrôle d'unicité et validation	Faire valider les règles métier et importer le référentiel institutionnel
Accès par rôle et consentement	Sessions révocables, rôles en base et consentement révocable	Compléter la vérification d'e-mail, la récupération de compte et la recette des textes
Recherche et indicateurs	Annuaire filtré, agrégats et répartition par promotion	Mesurer latence et lisibilité sur un volume représentatif
Actualisation annuelle	Campagnes préparées et confirmations suivies en base	L'envoi réel d'e-mails et le processus d'invitation restent à réaliser
Architecture S2 React/TypeScript	Frontend effectif en Vite/JavaScript ; backend FastAPI	Écart de stack explicite ; ne pas confondre conception et réalisation

Comparaison avant / projection To-Be

Les cibles ci-dessous sont celles du cadrage, à six mois après lancement. Elles sont des objectifs de projection, pas des gains déjà constatés. Les valeurs initiales doivent être recueillies avant de calculer une amélioration.

KPI	Avant	Projection To-Be	Formule métier
Alumni recensés	Non mesuré	Au moins 70 %	Profils uniques validés / diplômés ciblés × 100
Profils complets	Non mesuré	Au moins 85 %	Profils validés complets / profils validés × 100
Profils à jour	Non mesuré	Au moins 75 %	Profils confirmés depuis moins de 12 mois / profils validés × 100
Délai médian de validation	Non mesuré	Au plus 3 jours ouvrés	Médiane des durées soumission-validation selon le calendrier ouvré

Limites des indicateurs actuels

La couverture affichée repose sur les fiches recensées, faute de référentiel complet des diplômés ciblés. Le serveur calcule les délais en jours calendaires et la fraîcheur à 365 jours. Le compteur de complétude vérifie le nom et le matricule. Il faut aligner ces conventions avec les champs obligatoires et les jours ouvrés du cadrage avant un reporting institutionnel.

Collecte proposée : Service Alumni responsable de la référence initiale ; suivi mensuel de couverture et fraîcheur, hebdomadaire de complétude et des délais. Chaque extraction doit conserver sa date, sa population et ses dénominateurs.

2. Retour d’expérience

Du prototype local au MVP connecté

Le dépôt conserve une première démonstration locale et une version connectée. La version principale charge connected.js et appelle l’API. Les données métier sont stockées en base, indépendamment du navigateur. Le principal enjeu consiste à faire respecter les mêmes règles dans l’interface, les endpoints et les transitions de statut.

Difficulté	Réponse technique	Limite persistante
Droits de l’utilisateur	Rôle issu du compte en base, contrôle serveur et propriété du profil	Validation complète des parcours métiers encore nécessaire
Qualité des profils	Champs validés, unicité des identifiants, examen des doublons et revue humaine	Pas de rapprochement métier validé sur un référentiel réel
Consentement	Octroi/révocation datés ; annuaire limité aux profils publiables	Textes et politique de conservation à valider par l’établissement
Traçabilité et imports	Auteur et date des décisions, bilan CSV par ligne, exports filtrés	Pas de preuve de déploiement ou d’exploitation en production
Écart de conception	Réalisation effective documentée plutôt que stack annoncée fictivement	React/TypeScript et refresh token prévus en S2 non réalisés

Vérification technique disponible

10 tests backend réussis le 27 octobre 2026 sur le code de référence : identité, contrôle des rôles, propriété des fiches, workflow connecté, consentement, sessions, campagnes et import/export. Le lancement a émis 43 avertissements de dépréciation des dépendances. Ils ne sont pas des échecs de test, mais justifient une revue de maintenance.

Le bilan du dépôt du 22 octobre annonce 13 tests frontend, ESLint, Ruff et une compilation Vite réussie. Ces résultats sont documentaires : ils n’ont pas été réexécutés dans le contrôle du 27 octobre. Docker/PostgreSQL, montée en charge, sauvegarde et restauration restent à vérifier.

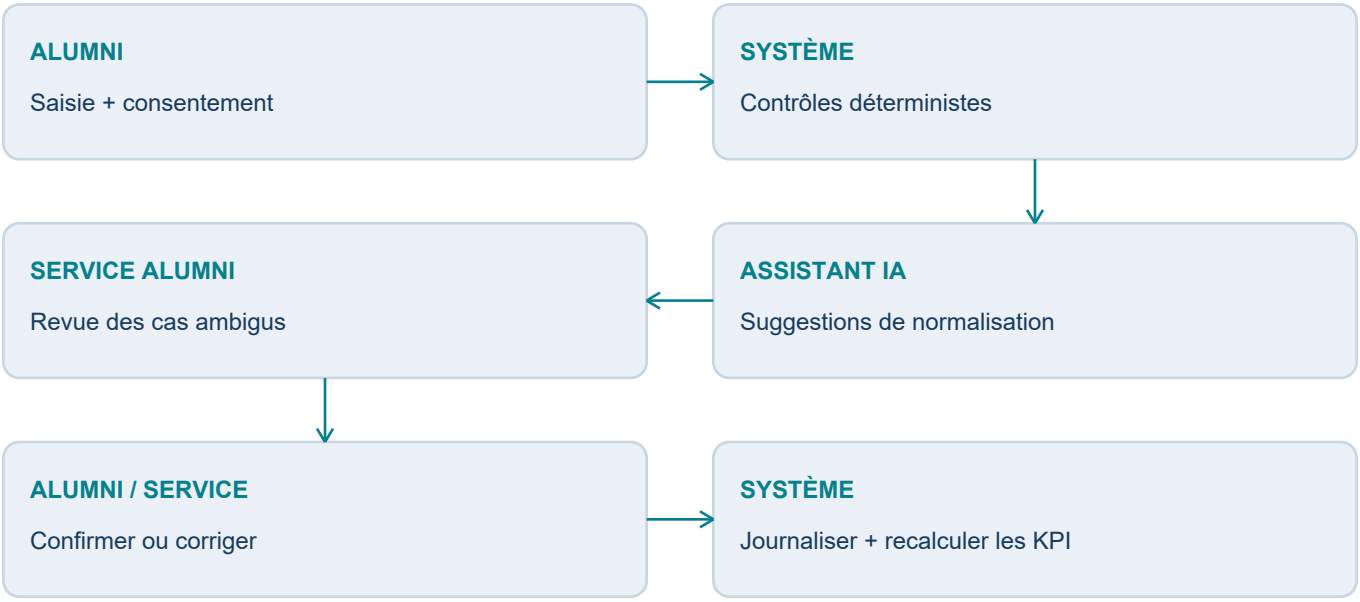
Apports et limites du développement assisté par IA

L’assistance peut accélérer le découpage des tâches, la génération du socle et l’écriture de tests. Elle exige une validation humaine des hypothèses, des règles d’accès et de la définition des KPI. Le contrôle par parcours et les tests métier permettent de repérer des écarts qu’une interface convaincante ne montre pas.

La comparaison Gemini / Auto-Standard / Grok n’est pas mesurable avec les preuves disponibles. Les vidéos des autres membres et les journaux complets d’exécution sont indisponibles. Aucun temps gagné ni classement de moteurs n’est inventé. Pour une comparaison reproductible, conserver le même point de départ, les prompts, plans validés, corrections, tests et commits finaux.

3. To-Be augmenté par l’IA

Processus proposé pour une évolution ultérieure, hors MVP actuel.



Répartition des responsabilités

Les contrôles de formats, champs obligatoires, identifiants et droits restent déterministes. L’assistant IA propose une normalisation des métiers, secteurs ou entreprises. Il ne modifie pas silencieusement la donnée d’origine et ne fusionne pas de personnes. Les cas ambigus sont revus par le Service Alumni ; toute décision conserve un auteur, une date et la proposition initiale.

Après validation, le système met à jour l’annuaire et les indicateurs. Une aide à la segmentation peut proposer les profils à relancer. Le Service Alumni approuve la campagne avant tout envoi et contrôle le respect du consentement.

Valeur métier et conditions de mesure

Évolution proposée	Valeur attendue	Indicateur de pilote
Normalisation assistée	Réduire la reprise manuelle des libellés	Temps médian de revue ; taux de propositions acceptées
Repérage des cas ambigus	Concentrer l’expertise humaine sur les exceptions	Faux rapprochements sur un échantillon annoté
Suggestion de relances	Améliorer l’actualisation sans sollicitation excessive	Confirmations après relance sur la même population

Garde-fous du processus

Conserver les données sources, rendre les corrections réversibles, minimiser les champs transmis et journaliser l’origine des suggestions. Évaluer les erreurs et les coûts sur un pilote avant extension. L’IA n’attribue ni consentement, ni identité définitive, ni statut de publication à la place des responsables.

4. Sources et architecture

Architecture logicielle effective

Couche	Technologie / responsabilité
Interface	Vite et JavaScript. connected.js appelle les endpoints /api.
API	FastAPI et Pydantic : validation des entrées, routes et contrôle des rôles.
Persistance	SQLAlchemy ; SQLite en local. PostgreSQL prévu pour l'environnement Docker.
Authentification	JWT en cookie HttpOnly ; session serveur révocable et rôles stockés en base.
Qualité / livraison	Pytest, tests frontend, Ruff/ESLint, Vite et workflow GitHub Actions.

Organisation du dépôt consolidé

Source	Responsabilité
frontend/public/connected.js	Parcours connectés, appels API et gestion des erreurs
frontend/public/domain.js	Utilitaires partagés et traitement CSV
backend/app/api.py	Workflow, annuaire, campagnes, KPI, import/export
backend/app/models.py et schemas.py	Tables relationnelles et schémas de validation
backend/app/auth.py et bootstrap.py	Sessions et création des comptes privilégiés
backend/migrations et tests	Évolution du schéma et vérifications automatisées
docker-compose.yml et .github/workflows	Services Docker prévus et contrôles automatisés
README.md, rapports et demo/	Installation, conception et vidéo existante de Ziad

Limites avant exploitation

Le MVP ne réalise pas de SSO, de récupération de mot de passe, de refresh token ni d'envoi réel d'e-mails. Les profils importés restent en attente d'un processus d'invitation non terminé. Une mise en production doit vérifier le déploiement PostgreSQL, HTTPS/cookie sécurisé, une clé de signature stable, l'anti-abus, les sauvegardes et un test de restauration.

Référence de ce rapport

Dépôt public : github.com/ziad4222/eigsi-suivi-alumni
Code évalué : 8fc289b ; ajout documentaire S1 : 8ac4282.
Les captures et les tests utilisent des données fictives. Les recommandations IA décrivent une évolution, sans annoncer une fonction déjà livrée.

4. Installation et déploiement

Prérequis documentés : Git, Node.js 22, pnpm et Python 3.12.

Démarrer l’API - PowerShell

```
git clone https://github.com/ziad4222/eigsi-suivi-alumni.git
cd eigsi-suivi-alumni
Copy-Item .env.example .env
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
alembic upgrade head
python -m app.bootstrap --demo
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Le bootstrap demande un mot de passe choisi localement et crée des comptes fictifs. Il n’est pas nécessaire de publier ce mot de passe. Le fichier .env reste hors du dépôt.

Démarrer l’interface - second terminal

Depuis la racine du dépôt :

```
cd frontend
pnpm install --frozen-lockfile
pnpm run dev
```

Interface : http://localhost:4173
Documentation API : http://localhost:8000/docs
Contrôle de santé : http://localhost:8000/health

Vérifier et préparer Docker

Contrôles : pnpm run lint, pnpm test, pnpm run build dans frontend ; ruff check . et pytest -q dans backend. Pour Docker, renseigner les variables PostgreSQL et le secret dans .env, puis utiliser docker compose up --build et initialiser les comptes avec docker compose exec backend python -m app.bootstrap --demo. Cette voie reste à tester ; sa présence dans le dépôt n’est pas une preuve de déploiement réussi.

Bilan et prochaines vérifications

Le socle permet de remplacer des échanges dispersés par un workflow centralisé et traçable. Les preuves techniques disponibles soutiennent les règles essentielles du backend. La recette client, la référence initiale des KPI, les vidéos manquantes et le déploiement restent à compléter pour conclure sur l’efficacité réelle du système.

Sources

Consignes Moodle : activité 93022, cours Information Systems.
Rapports S1 et S2 à la racine du dépôt ; README.md ; docs/SEANCES_3_4.md ; sources API et interface ; tests backend exécutés le 27/10/2026. Figures : docs/screenshots/dashboard-direction.png.